

Graduation Project Detailed Notes

DermaVision

منصة ويب ذكية مساعدة في مجال الجلدية تجمع بين تشخيص مبدئي من صورة جلدية، ومساعد RAG آمن، وتقارير قابلة للطباعة، مع بنية Backend/Frontend جاهزة للعرض والنشر.

Secure Full Stack

JWT auth, SQLite/Alembic,
Docker, reports, and
.history

Safe RAG Assistant

Arabic/English chatbot
grounded in dermatology
.knowledge

AI Diagnosis

EfficientNet-B4 predicts
the skin condition and
.confidence

1. الفلو العام للمشروع

الفكرة الأساسية أن الموديل يتوقع المرض من الصورة، ثم **RAG** يشرح النتيجة للمستخدم بطريقة تعليمية وآمنة.

1 تسجيل أو دخول

المستخدم يعمل حساب أو يسجل دخول، ويحصل على JWT Token.

2 رفع صورة الجلد

الواجهة ترسل الصورة للـ Backend مع التوكن.

3 تحقق من الصورة

الـ Backend يتحقق من النوع والحجم وأن الملف صورة حقيقية.

4 تشخيص مبدئي

EfficientNet-B4 يرجع predicted label وconfidence وtop predictions.

5 حفظ النتيجة

النظام يحفظ التشخيص والصورة والـ metadata في قاعدة البيانات.

6 عرض التقرير

المستخدم يرى النتيجة، جودة الصورة، الاحتمالات، ومعلومات عامة.

7 اختيار الخطوة التالية

إما تحميل Preliminary Report أو التحدث مع المساعد.

8 RAG Chat + Consultation Report

المساعد يجيب من قاعدة المعرفة، وبعد الحوار يمكن إنشاء Consultation Report.

مهم في المناقشة: النظام لا يدعي تشخيصًا نهائيًا. كل النتائج مبدئية وتعليمية، ويجب تأكيدها مع طبيب جلدية.

2. Project Overview

- **DermaVision** منصة Full-Stack لتحليل صور الجلد بالذكاء الاصطناعي.
- المستخدم يرفع صورة جلدية ويحصل على نتيجة مبدئية مع نسبة ثقة وجودة الصورة.
- النظام يوفر مساعد RAG يشرح الحالة، العدوى، الخطورة، العناية العامة، والأسئلة المناسبة للطبيب.
- يوجد نظام تقارير: Preliminary Report من صفحة النتيجة، و Consultation Report بعد المحادثة.
- كل المحتوى الطبي تعليمي ومساعد فقط، وليس بديلًا عن الطبيب.

3. لماذا اخترنا FastAPI في ال Backend؟

Swagger تلقائي

يعطي Documentation تلقائي على `/docs` لتجربة endpoints في المناقشة.

مناسب للـ AI APIs

FastAPI ممتاز في بناء APIs سريعة حول ML inference، خصوصًا مع upload files و async endpoints.

Async Support

مفيد مع database async، file handling، وعمليات RAG أو email.

Pydantic Validation

يساعد في request/response validation وتقليل أخطاء البيانات.

أكثر تنظيمًا من Flask للمشروع ده

Flask بسيط، لكن FastAPI أعطانا typing، validation، docs، dependency injection بشكل جاهز.

أخف من Django

Django قوي جدًا لكنه أكبر من المطلوب هنا، لأن المشروع API-focused وليس CMS أو Admin-heavy.

Architecture / Structure .4

ال Backend المعمول بأسلوب **Layered / Clean Architecture** بحيث كل طبقة لها مسؤولية واضحة. هذا يجعل المشروع أسهل في الشرح، الاختبار، والتطوير.



Backend Layers

- **API Layer:** ملفات `app/api/v1/endpoints` تستقبل الطلبات وترجع `responses`.
- **Domain/Service Layer:** ملفات مثل `diagnosis_service.py` و `rag_service.py` تنفذ الفلو والمنطق.
- **Infrastructure Layer:** ML model, RAG modules, email service, database session, providers
- **Database Layer:** SQLAlchemy models و repositories مثل `DiagnosisRepository`.

Frontend Structure

- **Pages:** Home, Upload, Result, Chat, Dashboard, History, Auth pages
- **Components:** Navbar, Footer, Hero, disease cards, family manager, etc
- **Services:** authService, diagnosisService, chatService, familyService
- **Contexts:** Auth, Language, Theme
- **Localization:** Arabic/English text and RTL support

Design Patterns Used .5

Service Layer Pattern

مثال: `DiagnosisService` يدير رفع الصورة،
prediction، والحفظ. `RagService` يدير
.chat/report flow

Layered / Clean Architecture

فصلنا API عن business logic عن infrastructure،
عشان أي جزء يتغير بدون كسر باقي النظام.

DTO / Mapper Pattern

mappers تحول database objects إلى API
response shape ثابت ومناسب للـ Frontend.

Repository Pattern

مثال: `DiagnosisRepository` يخفي تفاصيل
SQLAlchemy ويقدم functions مثل
.create/get_by_user

Fallback / Resilience Pattern

لو LLM provider فشل أو API key غير موجود،
النظام يرجع لإجابات deterministic آمنة بدل ما
ينهار.

Provider / Strategy Pattern

RAG يمكنه التبديل بين `groq` أو `xai` أو
`openai` أو local fallback بدون تغيير endpoint.

Middleware Pattern

Security headers، logging، و rate limiting يتم
تطبيقهم على الطلبات مركزيًا.

Centralized Configuration

كل الإعدادات مثل DB URL و CORS و LLM
providers و SMTP تأتي من environment
variables.

AI Diagnosis .6

- الموديل المستخدم: **EfficientNet-B4**.
- ال inference يتم في Backend باستخدام PyTorch/timm.
- الموديل لا يشرح المرض؛ دوره فقط prediction من الصورة.
- RAG هو المسؤول عن شرح النتيجة للمستخدم.

Supported Classes

Mycosis Fungoides / MF

Psoriasis

Annular Lichen / Lichen Planus

Tinea Circinata

Urticaria

Healthy

Diagnosis Output

- predicted_label : اسم الحالة المتوقعة.
- confidence : نسبة ثقة الموديل.
- all_predictions / top_k : أعلى احتمالات.
- image_quality_score : تقييم جودة الصورة من 0 إلى 100.
- analysis_id : رقم التحليل المستخدم في RAG chat history وreports.

7. قاعدة البيانات: كم Database؟ وما الجداول؟

عندنا **Database** واحدة للتطبيق في التشغيل المحلي/demo وهي SQLite file غالبًا باسم `skin_analysis.db` . في الإنتاج يمكن استبدالها بـ PostgreSQL باستخدام نفس SQLAlchemy/Alembic setup. الجداول الرئيسية في التطبيق: **4 application tables**، بالإضافة إلى جدول Alembic داخلي لتتبع migrations.

Table: users

Purpose	Type	Column
رقم المستخدم الأساسي.	Integer PK	id
بريد المستخدم لتسجيل الدخول.	String(255), unique	email
اسم المستخدم المعروض.	String(100), unique	username
كلمة المرور بعد hashing، لا نخزن password صريح.	String(255)	hashed_password
حالة الحساب.	Boolean	is_active
وقت إنشاء الحساب.	DateTime	created_at
آخر تحديث للحساب.	DateTime	updated_at

Table: diagnoses

Purpose	Type	Column
رقم التحليل / analysis id.	Integer PK	id
ربط التشخيص بصاحب الحساب.	FK to users.id	user_id
اختياري: ربط التحليل بفرد من العائلة.	FK nullable	family_member_id
معرف جلسة آمن لتسمية الصورة وتتبع التحليل.	String(36), unique	session_id
مسار الصورة المحفوظة محليًا.	String(512)	image_path
جودة الصورة من 0 إلى 100.	Integer	image_quality_score
تصنيف الجودة مثل Good / Poor.	String(50)	image_quality_label
اسم الحالة المتوقعة من الموديل.	String(255)	disease_type
ثقة الموديل كنسبة 0 إلى 1.	Float	probability
الثقة كنسبة مئوية.	Float	confidence_percentage
كل التوقعات المحفوظة كـ JSON text.	Text JSON	all_predictions
بيانات إضافية عن التحليل أو المعالجة.	Text JSON	extra_metadata
وقت التحليل.	DateTime	created_at
آخر تحديث.	DateTime	updated_at

Table: family_members

Purpose	Type	Column
رقم فرد العائلة.	Integer PK	id
صاحب الحساب.	FK to users.id	user_id
اسم الفرد.	String(100)	name
العلاقة: self, child, parent, spouse, sibling, other.	String(50)	relation
تاريخ الميلاد اختياري.	Date nullable	date_of_birth
النوع اختياري.	String(10) nullable	gender
ملاحظات بسيطة.	String(500) nullable	notes
وقت الإضافة.	DateTime	created_at
آخر تحديث.	DateTime	updated_at

Table: password_reset_tokens

Purpose	Type	Column
رقم token record.	Integer PK	id
المستخدم صاحب طلب إعادة التعيين.	FK to users.id	user_id
النسخة hashed من reset token: لا يتم تخزين raw token.	String(64), unique	token_hash
وقت انتهاء صلاحية token.	DateTime	expires_at
لو استخدم token يصبح single-use.	DateTime nullable	used_at
وقت إنشاء token.	DateTime	created_at

جدول alembic_version ليس جدول business data، لكنه موجود لتتبع آخر migration تم تطبيقه.

8. لماذا SQLite محليًا؟ ولماذا Alembic؟

SQLAlchemy يحافظ على قابلية النقل

لأننا نستخدم ORM، يمكن الانتقال إلى PostgreSQL في الإنتاج بدون إعادة كتابة كل logic.

SQLite مناسب للعرض والتطوير

لا يحتاج Database server منفصل، وهذا يسهل تشغيل المشروع في viva أو Docker demo.

PostgreSQL أفضل للإنتاج

لأن الإنتاج يحتاج concurrency أعلى، backups، مراقبة، وأداء أقوى من SQLite.

Alembic لإدارة schema

يحفظ تطور الجداول عبر migrations بدل إنشاء الجداول يدويًا.

9. RAG Architecture: ماذا أضفنا ولماذا؟

RAG في DermaVision ليس Chat عام. هو Diagnosis-aware assistant: يبدأ من `analysis_id` ، يتأكد من ملكية المستخدم للتشخيص، ثم يستخدم predicted disease لتوجيه البحث والإجابة.

RAG Flow



RAG Modules

Role	Module
تحميل ملف المعرفة skin_data1.txt .	document_loader.py
تقسيم النص إلى chunks مناسبة للبحث.	text_splitter.py
فهرس محلي lightweight يشبه TF-IDF ويخزن disease tags.	vector_store.py
استرجاع chunks مرتبطة بالمرض والنية، مع reranking.	retriever.py
كشف intents محليًا: diagnosis, treatment, contagion, urgent, dosage, age groups, etc.	intent.py
تصنيف intent بال LLM اختياريًا لو provider متاح.	semantic_intent_classifier.py
Fallback deterministic generator لإجابات آمنة بدون API keys.	generator.py
canonical names وحقائق safety حتمية لكل مرض.	disease_profiles.py
Provider strategy لاستخدام LLM اختياريًا.	xai_generator.py , groq_generator.py , openai_generator.py
ينسق كل الفلو: diagnosis context, retrieval, intent, generation, validation, final report.	rag_service.py

لماذا أضفنا RAG؟

- الموديل يتوقع label فقط، ولا يشرح المرض للمستخدم.
- RAG يجعل الشرح grounded في ملف معرفة محلي بدل إجابة عامة من LLM.
- RAG يقلل hallucination لأن الإجابة مرتبطة بمصادر وسياق تشخيص المستخدم.
- أضفنا deterministic safety profiles لأن بعض الحقائق الطبية لا يجب تركها لل LLM.

Medical Safety Guardrails .10

- لا توجد وصفة شخصية، ولا جرعات، ولا عدد مرات استخدام، ولا مدة علاج شخصية.
- أسئلة الخطورة مثل "ممكن أموت؟" تتحول إلى urgent-care intent.
- Urticaria يتم إجبارها كحالة غير معدية.
- Tinea يتم إجبارها كحالة معدية لأنها عدوى فطرية.
- canonical disease names تمنع أخطاء ترجمة مثل ترجمة Urticaria بشكل خاطئ.
- post-generation validation يرفض عبارات خطرة أو غير دقيقة من LLM.
- disclaimer أن النظام للتوعية ولا يغني عن طبيب جلدية.

الفكرة المهمة: LLM لا يأخذ القرار الطبي وحده. الحقائق عالية الخطورة تأتي من disease_profiles.
deterministic rules

Security Features .11

Protected Endpoints

ال API يعتمد على `get_current_user_id` لحماية البيانات.

JWT Authentication

التوكن يرسل مع الطلبات المحمية مثل `.diagnosis/history/RAG`.

Password Hashing

كلمات المرور تحفظ hashed فقط.

Ownership Checks

المستخدم لا يستطيع فتح `diagnosis` أو `report` لا يخصه.

Upload Security

10MB limit، MIME فقط، JPG/JPEG/PNG validation، وPillow image verification.

Password Reset Tokens

reset token يتم تخزينه single-use، hashed، وله expiry.

Security Headers

middleware يضيف headers تقلل مخاطر browser-side attacks.

Rate Limiting

مطبق على `login/signup/forgot/reset/diagnosis/RAG.chat`.

Secrets Hygiene

API keys وSMTP/JWT secrets في backend env فقط، و `.env` خارج Git.

CORS

السماح فقط بال frontend origins المطلوبة.

Report System .12

Preliminary Report

- ينشأ من صفحة نتيجة التشخيص.
- يعتمد على AI diagnosis وال basic report الموجود في الصفحة.
- قابل للطباعة أو الحفظ PDF.
- لا يسمى final diagnosis ولا personal prescription.

Consultation Report

- ينشأ من صفحة chat بعد المحادثة.
- يرسل chat messages + analysis_id إلى POST /api/v1/rag/final-report .
- يتضمن summary، diagnosis context، لأسئلة المريض، red flags، general care guidance، و disclaimer.
- لا يظهر كتشخيص نهائي؛ هو تقرير توعوي للاستشارة.

Frontend Features .13

- React/Vite frontend مع API client مركزي مبني على Axios.
- Arabic/English localization و RTL handling للعربي.
- Auth pages: sign in, sign up, forgot password, reset password
- Upload page مع validation ونظام crop/preview.
- Result page يدعم refresh عن طريق `/result?id=<analysisId>` .
- Dashboard و History لعرض التحليلات السابقة.
- Family members support لاختيار الشخص الذي يتم التحليل له.
- Chat page يحفظ المحادثة لكل diagnosis في browser session storage.
- Sources مخفية خلف View sources حتى لا تكون الواجهة تقنية جدًا.
- Dark medical UI design، مع Home page صور DermaVision الجديدة.

DevOps / GitHub Readiness .14

- Backend Dockerfile لتشغيل FastAPI على port 8000.
- Frontend Dockerfile مع nginx لخدمة Vite build.
- root docker-compose.yml لتشغيل backend + frontend معًا.
- .dockerignore و .gitignore. يمنعان رفع الملفات المولدة أو الأسرار.
- README files محدثة للتشغيل المحلي و Docker و بيئة الإنتاج.
- ملف الموديل حجمه تقريبًا 67.7 MiB؛ Git LFS مناسب أكثر لو الريبو public ومنظم.
- لا يتم رفع .env أو real API keys أو SMTP password.

What Makes the Project Strong .15

- End-to-end AI workflow: من رفع الصورة إلى التشخيص والتقرير.
- Real ML inference وليس mock prediction.
- RAG chatbot آمن ومربوط بنتيجة التشخيص الحقيقية.
- يدعم أسئلة multi-intent بالعربي والإنجليزي.
- Clean architecture و design patterns واضحة يمكن شرحها في المناقشة.
- Authentication, history, family members, password reset.
- تقارير printable ومناسبة للاستشارة.
- Docker/GitHub readiness يجعل المشروع قابل للتشغيل عند أي مطور.

Short Presentation Summary .16

DermaVision is a full-stack AI-assisted dermatology system. It combines image-based skin disease prediction, secure user management, diagnosis history, a safe RAG medical assistant, and printable reports. The backend follows layered clean architecture with service, repository, DTO, middleware, and provider strategy patterns. The system supports Arabic and English, Docker-based setup, optional Groq/xAI/OpenAI generation, and deterministic medical safety guardrails so the output remains educational and not a personal prescription.

Final viva sentence: "Model predicts, RAG explains, and safety guardrails control the medical wording."